**APEX RETAIL GROUP | PRACTICAL CLASS TASKS**

**DOCUMENT:** CLASS TASKS (SESSION 5)

**SCENARIO:** CUSTOMER RETURNS DATA SANITIZATION & QUALITY AUDIT

**INSTRUCTIONS FOR LEARNER**: Complete the following four tasks in your Jupyter Notebook during class. Follow the structured workflow: **Defect Identification -> Deduplication -> String & Date Cleaning -> Missing Value Imputation -> Executive Reporting**. Do not peek at instructor solution keys until completed.

## TASK 1: DEFECT IDENTIFICATION & DATASET LOADING

Rachel Torres (Customer Success Lead) provides a messy export log of recent customer returns:

```python
import pandas as pd
import numpy as np

raw_returns_data = {
    "return_id": ["RET-301", "RET-302", "RET-303", "RET-303", "RET-304", "RET-305", "RET-306", "RET-307"],
    "log_date": ["2026-10-01", "2026-10-01", "2026-10-02", "2026-10-02", "2026-10-03", "2026-10-03", "2026-10-04", "2026-10-04"],
    "item_category": [" APPAREL ", "Electronics", " home ", " home ", "Apparel", " ELECTRONICS ", "Apparel", "Home"],
    "refund_usd": [45.00, 299.00, np.nan, np.nan, 60.00, 150.00, np.nan, 85.00],
    "customer_rating": [1, 2, 1, 1, 3, 2, 1, 4]
}
```

1. Import `pandas` as `pd` and `numpy` as `np`. Load `raw_returns_data` into DataFrame `df_returns`.
2. Display the initial shape (`.shape`) of `df_returns`.
3. Count missing values per column using `.isnull().sum()`.
4. Count total exact duplicate rows using `.duplicated().sum()`.

## TASK 2: DEDUPLICATION & STRING CLEANING

1. Remove exact duplicate rows using `.drop_duplicates()` and reset the index (`reset_index(drop=True)`).
2. Clean the `item_category` column:
    - Use `.str.strip()` to strip leading and trailing whitespace.
    - Use `.str.title()` to standardize casing to Title Case.

3. Print the unique values in `item_category` using `.unique()` to verify that categories are clean and unified.

## TASK 3: DATE PARSING & MISSING VALUE IMPUTATION

1. Convert the `log_date` column from string text to explicit datetime format using `pd.to_datetime()`. Verify using `df_returns.dtypes`.
2. Calculate the median value of the `refund_usd` column across non-missing entries.
3. Impute missing values in `refund_usd` using `.fillna()` with the calculated median refund value.
4. Verify that `df_returns.isnull().sum()` shows zero missing values across all columns.

## TASK 4: DATA QUALITY IMPACT MEMO

Review your cleaned DataFrame:

1. Compare total refund cost (`refund_usd.sum()`) BEFORE imputation versus AFTER imputation.
2. How many duplicate records were eliminated?
3. Draft a 2-3 sentence data quality memo to Rachel Torres explaining why median imputation was preferred over dropping rows with missing refund values.